Laboratório da Metodo de Conjuntos

[2]:
```
import numpy as np
import matplotlib.pyplot as plt
from scipy.special import comb
from scipy.stats import mode
from sklearn.datasets import make_moons, load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import BaggingClassifier, RandomForestClassifier,␣
 ↪ExtraTreesClassifier
from sklearn.metrics import accuracy_score
from sklearn.inspection import permutation_importance
import time

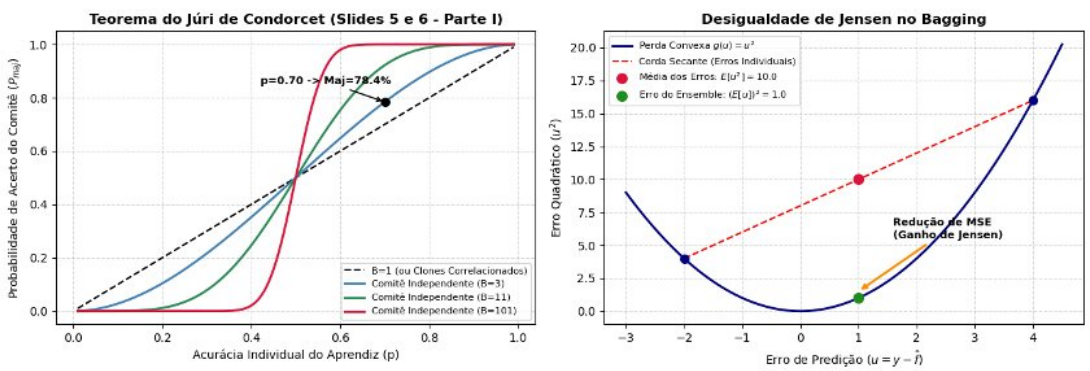
def prob_condorcet(B, p):
    min_votos = (B // 2) + 1
    return sum(comb(B, k) * (p ** k) * ((1.0 - p) ** (B - k)) for k in␣
 ↪range(min_votos, B + 1))

p_vals = np.linspace(0.01, 0.99, 200)

fig, axes = plt.subplots(1, 2, figsize=(13, 4.5))
axes[0].plot(p_vals, p_vals, 'k--', linewidth=1.5, label='B=1 (ou Clones␣
 ↪Correlacionados)')
for B_tam, cor in [(3, 'steelblue'), (11, 'seagreen'), (101, 'crimson')]:
    curva_maj = [prob_condorcet(B_tam, p) for p in p_vals]
    axes[0].plot(p_vals, curva_maj, color=cor, linewidth=2, label=f'Comitê␣
 ↪Independente (B={B_tam})')
axes[0].scatter([0.70], [prob_condorcet(3, 0.70)], color='black', zorder=5,␣
 ↪s=50)
axes[0].annotate("p=0.70 -> Maj=78.4%", xy=(0.70, 0.784), xytext=(0.42, 0.85),
                 arrowprops=dict(arrowstyle="->", lw=1.2), fontweight='bold',␣
 ↪fontsize=9)
axes[0].set_title("Teorema do Júri de Condorcet (Slides 5 e 6 - Parte I)",␣
 ↪fontweight='bold')
axes[0].set_xlabel("Acurácia Individual do Aprendiz (p)")
axes[0].set_ylabel("Probabilidade de Acerto do Comitê ($P_{maj}$)")
axes[0].legend(loc='lower right', fontsize=8)
```

```
axes[0].grid(True, linestyle='--', alpha=0.5)

u_vals = np.linspace(-3.0, 4.5, 200)
u1, u2 = 4.0, -2.0
media_quadrados = (u1**2 + u2**2) / 2.0
erro_medio = (u1 + u2) / 2.0
quadrado_media = erro_medio ** 2
axes[1].plot(u_vals, u_vals**2, color='navy', linewidth=2, label='Perda Convexa␣
 ↪$g(u) = u^2$')
axes[1].plot([u2, u1], [u2**2, u1**2], 'r--', linewidth=1.5, label='Corda␣
 ↪Secante (Erros Individuais)')
axes[1].scatter([u2, u1], [u2**2, u1**2], color='navy', s=50, zorder=5)
axes[1].scatter([erro_medio], [media_quadrados], color='crimson', s=70,␣
 ↪zorder=6, label=f'Média dos Erros: $E[u^2] = {media_quadrados:.1f}$')
axes[1].scatter([erro_medio], [quadrado_media], color='forestgreen', s=70,␣
 ↪zorder=6, label=f'Erro do Ensemble: $(E[u])^2 = {quadrado_media:.1f}$')
axes[1].annotate("Redução de MSE\n(Ganho de Jensen)", xy=(erro_medio,␣
 ↪quadrado_media + 0.5), xytext=(1.6, 5.5),
                 arrowprops=dict(arrowstyle="->", color='darkorange', lw=2),␣
 ↪fontweight='bold', fontsize=9)
axes[1].set_title("Desigualdade de Jensen no Bagging", fontweight='bold')
axes[1].set_xlabel("Erro de Predição ($u = y - \\hat{f}$)")
axes[1].set_ylabel("Erro Quadrático ($u^2$)")
axes[1].legend(loc='upper left', fontsize=8)
axes[1].grid(True, linestyle='--', alpha=0.5)
plt.tight_layout()
plt.show()
```



[9]:
```
class BaggingClassifierNumPy:
    def __init__(self, n_estimators=100, voting='hard', random_state=42):
        self.n_estimators = n_estimators
        self.voting = voting
        self.random_state = random_state
```

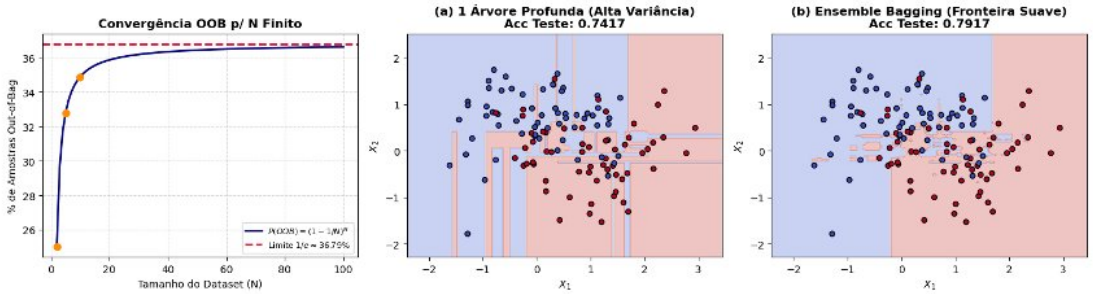
```
        self.estimators_ = []
        self.oob_indices_ = []
    def fit(self, X, y):
        np.random.seed(self.random_state)
        N = X.shape[0]
        self.estimators_ = []
        self.oob_indices_ = []
        for b in range(self.n_estimators):
            boot_idx = np.random.choice(N, size=N, replace=True)
            oob_idx = np.setdiff1d(np.arange(N), boot_idx)
            tree = DecisionTreeClassifier(random_state=self.random_state + b)
            tree.fit(X[boot_idx], y[boot_idx])
            self.estimators_.append(tree)
            self.oob_indices_.append(oob_idx)
        return self
    def predict_proba(self, X):
        all_probas = np.array([tree.predict_proba(X) for tree in self.
 ↪estimators_])
        return np.mean(all_probas, axis=0)
    def predict(self, X):
        if self.voting == 'soft':
            return np.argmax(self.predict_proba(X), axis=1)
        preds = np.array([tree.predict(X) for tree in self.estimators_])
        return mode(preds, axis=0, keepdims=False).mode

probas_slide15 = np.array([[0.01, 0.99], [0.55, 0.45], [0.55, 0.45]])
voto_hard_s15 = mode(np.argmax(probas_slide15, axis=1), keepdims=False).mode
voto_soft_s15 = np.argmax(np.mean(probas_slide15, axis=0))
print("=== HARD VS. SOFT VOTING ===")
print(f"Votos Discretos das 3 Árvores: {np.argmax(probas_slide15, axis=1)} ->␣
 ↪Decisão Hard Voting: Classe {voto_hard_s15}")
print(f"Probabilidade Média Classe 1 : {np.mean(probas_slide15[:, 1]):.2f}   ␣
 ↪-> Decisão Soft Voting: Classe {voto_soft_s15}\n")

X_moons, y_moons = make_moons(n_samples=400, noise=0.5, random_state=42)
X_m_tr, X_m_te, y_m_tr, y_m_te = train_test_split(X_moons, y_moons, test_size=0.
 ↪3, random_state=42)
arvore_unica = DecisionTreeClassifier(random_state=42).fit(X_m_tr, y_m_tr)
bag_moons = BaggingClassifierNumPy(n_estimators=100, voting='soft',␣
 ↪random_state=42).fit(X_m_tr, y_m_tr)
fig, axes = plt.subplots(1, 3, figsize=(15, 4.2))
N_finitos = np.arange(2, 101)
p_oob_exato = (1.0 - 1.0 / N_finitos) ** N_finitos * 100.0
axes[0].plot(N_finitos, p_oob_exato, color='navy', linewidth=2, label='$P(OOB)␣
 ↪= (1 - 1/N)^N$')
```

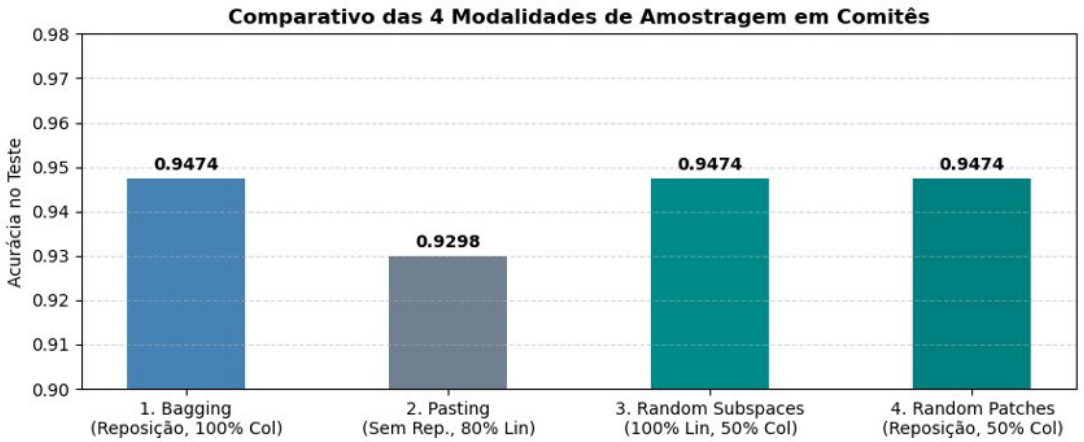
```
axes[0].axhline((1.0 / np.e) * 100.0, color='crimson', linestyle='--',␣
 ↪linewidth=2, label='Limite $1/e \\approx 36.79\\%$')
axes[0].scatter([2, 5, 10], [(1 - 1/n)**n * 100 for n in [2, 5, 10]],␣
 ↪color='darkorange', zorder=5, s=45)
axes[0].set_title("Convergência OOB p/ N Finito", fontweight='bold')
axes[0].set_xlabel("Tamanho do Dataset (N)")
axes[0].set_ylabel("% de Amostras Out-of-Bag")
axes[0].legend(loc='lower right', fontsize=8)
axes[0].grid(True, linestyle='--', alpha=0.5)
xx, yy = np.meshgrid(np.linspace(X_moons[:, 0].min() - 0.5, X_moons[:, 0].max()␣
 ↪+ 0.5, 180),
                     np.linspace(X_moons[:, 1].min() - 0.5, X_moons[:, 1].max()␣
 ↪+ 0.5, 180))
malha = np.c_[xx.ravel(), yy.ravel()]
axes[1].contourf(xx, yy, arvore_unica.predict(malha).reshape(xx.shape), alpha=0.
 ↪3, cmap='coolwarm')
axes[1].scatter(X_m_te[:, 0], X_m_te[:, 1], c=y_m_te, cmap='coolwarm',␣
 ↪edgecolors='k', s=25)
axes[1].set_title(f"(a) 1 Árvore Profunda (Alta Variância)\nAcc Teste:␣
 ↪{accuracy_score(y_m_te, arvore_unica.predict(X_m_te)):.4f}",␣
 ↪fontweight='bold')
axes[1].set_xlabel("$X_1$")
axes[1].set_ylabel("$X_2$")
axes[2].contourf(xx, yy, bag_moons.predict(malha).reshape(xx.shape), alpha=0.3,␣
 ↪cmap='coolwarm')
axes[2].scatter(X_m_te[:, 0], X_m_te[:, 1], c=y_m_te, cmap='coolwarm',␣
 ↪edgecolors='k', s=25)
axes[2].set_title(f"(b) Ensemble Bagging (Fronteira Suave)\nAcc Teste:␣
 ↪{accuracy_score(y_m_te, bag_moons.predict(X_m_te)):.4f}", fontweight='bold')
axes[2].set_xlabel("$X_1$")
axes[2].set_ylabel("$X_2$")
plt.tight_layout()
plt.show()
```

```
=== HARD VS. SOFT VOTING ===
Votos Discretos das 3 Árvores: [1 0 0] -> Decisão Hard Voting: Classe 0
Probabilidade Média Classe 1 : 0.63    -> Decisão Soft Voting: Classe 1
```



[10]:
```
cancer = load_breast_cancer()
X_tr, X_te, y_tr, y_te = train_test_split(
    cancer.data, cancer.target, test_size=0.30, random_state=42,␣
 ↪stratify=cancer.target
)
modalidades_slide13 = {
    "1. Bagging\n(Reposição, 100% Col)": BaggingClassifier(
        n_estimators=100, bootstrap=True, max_samples=1.0, max_features=1.0,␣
 ↪random_state=42, n_jobs=-1
    ),
    "2. Pasting\n(Sem Rep., 80% Lin)": BaggingClassifier(
        n_estimators=100, bootstrap=False, max_samples=0.8, max_features=1.0,␣
 ↪random_state=42, n_jobs=-1
    ),
    "3. Random Subspaces\n(100% Lin, 50% Col)": BaggingClassifier(
        n_estimators=100, bootstrap=False, max_samples=1.0, max_features=0.5,␣
 ↪random_state=42, n_jobs=-1
    ),
    "4. Random Patches\n(Reposição, 50% Col)": BaggingClassifier(
        n_estimators=100, bootstrap=True, max_samples=1.0, max_features=0.5,␣
 ↪random_state=42, n_jobs=-1
    )
}
nomes_mod, accs_mod = [], []
for nome, mod in modalidades_slide13.items():
    mod.fit(X_tr, y_tr)
    nomes_mod.append(nome)
    accs_mod.append(accuracy_score(y_te, mod.predict(X_te)))
plt.figure(figsize=(9, 3.8))
barras_mod = plt.bar(nomes_mod, accs_mod, color=['steelblue', 'slategray',␣
 ↪'darkcyan', 'teal'], width=0.45)
plt.ylim(0.90, 0.98)
plt.title("Comparativo das 4 Modalidades de Amostragem em Comitês",␣
 ↪fontweight='bold')
plt.ylabel("Acurácia no Teste")
```

```
for b in barras_mod:
    h = b.get_height()
    plt.text(b.get_x() + b.get_width()/2., h + 0.002, f"{h:.4f}", ha='center',␣
 ↪fontweight='bold')
plt.grid(axis='y', linestyle='--', alpha=0.5)
plt.tight_layout()
plt.show()
```

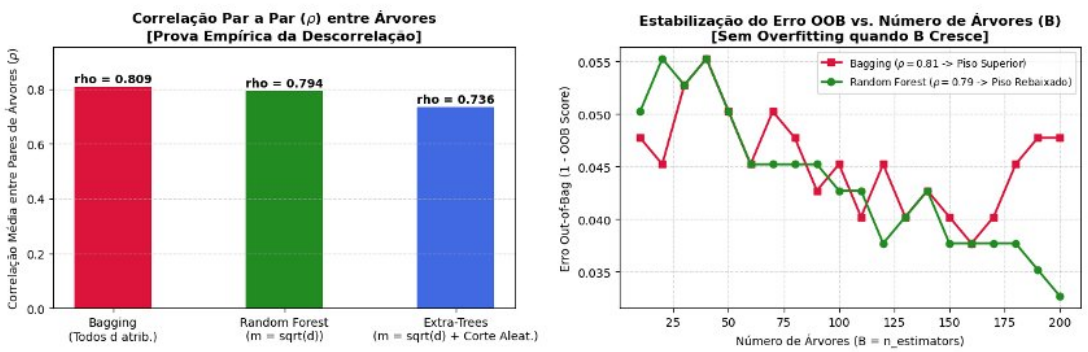


[13]:
```
def calcular_correlacao_media_arvores(ensemble_model, X_dados):
    preds_matriz = np.array([tree.predict(X_dados) for tree in ensemble_model.
 ↪estimators_])
    corr_matrix = np.corrcoef(preds_matriz)
    B = len(ensemble_model.estimators_)
    mascara_fora_diag = ~np.eye(B, dtype=bool)
    return np.mean(corr_matrix[mascara_fora_diag])

bag_100 = BaggingClassifier(n_estimators=100, random_state=42, n_jobs=-1).
 ↪fit(X_tr, y_tr)
rf_100 = RandomForestClassifier(n_estimators=100, max_features='sqrt',␣
 ↪random_state=42, n_jobs=-1).fit(X_tr, y_tr)
et_100 = ExtraTreesClassifier(n_estimators=100, max_features='sqrt',␣
 ↪bootstrap=True, random_state=42, n_jobs=-1).fit(X_tr, y_tr)
rho_bag = calcular_correlacao_media_arvores(bag_100, X_te)
rho_rf = calcular_correlacao_media_arvores(rf_100, X_te)
rho_et = calcular_correlacao_media_arvores(et_100, X_te)
b_lista = list(range(10, 201, 10))
erros_oob_bag, erros_oob_rf = [], []
for b in b_lista:
    m_bag = BaggingClassifier(n_estimators=b, oob_score=True, random_state=42,␣
 ↪n_jobs=-1).fit(X_tr, y_tr)
```

```
    erros_oob_bag.append(1.0 - m_bag.oob_score_)
    m_rf = RandomForestClassifier(n_estimators=b, max_features='sqrt',␣
 ↪oob_score=True, random_state=42, n_jobs=-1).fit(X_tr, y_tr)
    erros_oob_rf.append(1.0 - m_rf.oob_score_)
fig, axes = plt.subplots(1, 2, figsize=(13, 4.3))
barras_rho = axes[0].bar(['Bagging\n(Todos d atrib.)', 'Random Forest\n(m =␣
 ↪sqrt(d))', 'Extra-Trees\n(m = sqrt(d) + Corte Aleat.)'],
                         [rho_bag, rho_rf, rho_et], color=['crimson',␣
 ↪'forestgreen', 'royalblue'], width=0.45)
axes[0].set_ylim(0, max(rho_bag, rho_rf, rho_et) * 1.18)
axes[0].set_title("Correlação Par a Par ($\\rho$) entre Árvores\n[Prova␣
 ↪Empírica da Descorrelação]", fontweight='bold')
axes[0].set_ylabel("Correlação Média entre Pares de Árvores ($\\rho$)")

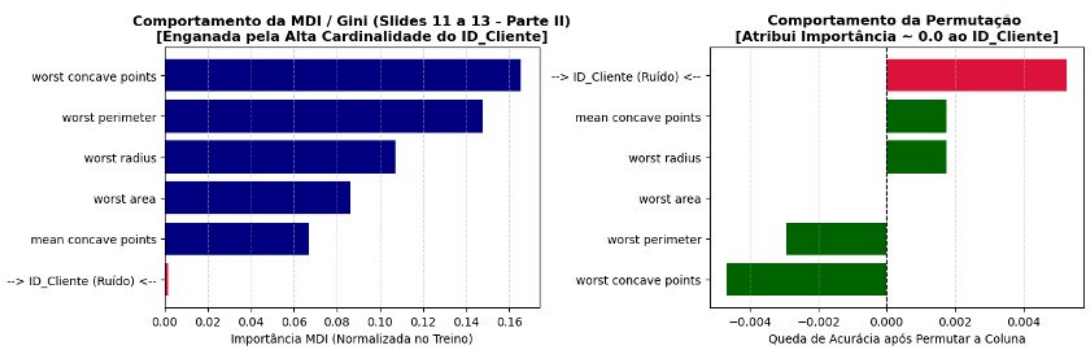
for b in barras_rho:
    h = b.get_height()
    axes[0].text(b.get_x() + b.get_width()/2., h + 0.015, f"rho = {h:.3f}",␣
 ↪ha='center', fontweight='bold')
    axes[0].grid(axis='y', linestyle='--', alpha=0.5)
axes[1].plot(b_lista, erros_oob_bag, marker='s', color='crimson', linewidth=2,␣
 ↪label=f'Bagging ($\\rho={rho_bag:.2f}$ -> Piso Superior)')
axes[1].plot(b_lista, erros_oob_rf, marker='o', color='forestgreen',␣
 ↪linewidth=2, label=f'Random Forest ($\\rho={rho_rf:.2f}$ -> Piso Rebaixado)')
axes[1].set_title("Estabilização do Erro OOB vs. Número de Árvores (B)\n[Sem␣
 ↪Overfitting quando B Cresce]", fontweight='bold')
axes[1].set_xlabel("Número de Árvores (B = n_estimators)")
axes[1].set_ylabel("Erro Out-of-Bag (1 - OOB Score)")
axes[1].legend(loc='upper right', fontsize=8.5)
axes[1].grid(True, linestyle='--', alpha=0.5)
plt.tight_layout()
plt.show()
```



[16]:
```
id_cliente_tr = np.arange(1, X_tr.shape[0] + 1).reshape(-1, 1)
id_cliente_te = np.arange(1, X_te.shape[0] + 1).reshape(-1, 1)
X_tr_id = np.hstack([X_tr, id_cliente_tr])
X_te_id = np.hstack([X_te, id_cliente_te])
nomes_com_id = list(cancer.feature_names) + ["--> ID_Cliente (Ruído) <--"]
rf_id = RandomForestClassifier(n_estimators=100, random_state=42, n_jobs=-1).
 ↪fit(X_tr_id, y_tr)
mdi_vals = rf_id.feature_importances_
perm_vals = permutation_importance(rf_id, X_te_id, y_te, n_repeats=10,␣
 ↪random_state=42, n_jobs=-1).importances_mean
idx_exibir = list(np.argsort(mdi_vals)[::-1][:5])
if 30 not in idx_exibir:
    idx_exibir.append(30)

idx_ord_mdi = sorted(idx_exibir, key=lambda i: mdi_vals[i])
idx_ord_perm = sorted(idx_exibir, key=lambda i: perm_vals[i])
fig, axes = plt.subplots(1, 2, figsize=(13, 4.2))
axes[0].barh([nomes_com_id[i] for i in idx_ord_mdi], [mdi_vals[i] for i in␣
 ↪idx_ord_mdi],
             color=['crimson' if i == 30 else 'navy' for i in idx_ord_mdi])
axes[0].set_title("Comportamento da MDI / Gini (Slides 11 a 13 - Parte␣
 ↪II)\n[Enganada pela Alta Cardinalidade do ID_Cliente]", fontweight='bold')
axes[0].set_xlabel("Importância MDI (Normalizada no Treino)")
axes[0].grid(axis='x', linestyle='--', alpha=0.5)
axes[1].barh([nomes_com_id[i] for i in idx_ord_perm], [perm_vals[i] for i in␣
 ↪idx_ord_perm],
             color=['crimson' if i == 30 else 'darkgreen' for i in␣
 ↪idx_ord_perm])
axes[1].axvline(0, color='black', linestyle='--', linewidth=1)
axes[1].set_title("Comportamento da Permutação \n[Atribui Importância ~ 0.0 ao␣
 ↪ID_Cliente]", fontweight='bold')
axes[1].set_xlabel("Queda de Acurácia após Permutar a Coluna")
axes[1].grid(axis='x', linestyle='--', alpha=0.5)

plt.tight_layout()
plt.show()
```

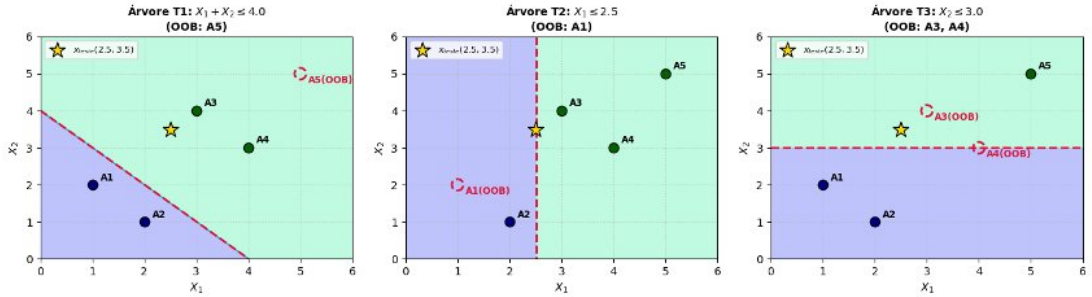


[24]:
```
X_manual = np.array([[1.0, 2.0], [2.0, 1.0], [3.0, 4.0], [4.0, 3.0], [5.0, 5.
↪0]])
y_manual = np.array([0, 0, 1, 1, 1])
nomes_A = ["A1(1,2)", "A2(2,1)", "A3(3,4)", "A4(4,3)", "A5(5,5)"]
T1_pred = lambda x: np.where((x[:, 0] + x[:, 1]) <= 4.0, 0, 1)
T2_pred = lambda x: np.where(x[:, 0] <= 2.5, 0, 1)
T3_pred = lambda x: np.where(x[:, 1] <= 3.0, 0, 1)
oob_de_cada_amostra = {0: [("T2", T2_pred)], 1: [], 2: [("T3", T3_pred)], 3:␣
↪[("T3", T3_pred)], 4: [("T1", T1_pred)]}
x_teste_manual = np.array([[2.5, 3.5]])
votos_teste = [int(T1_pred(x_teste_manual)[0]),␣
↪int(T2_pred(x_teste_manual)[0]), int(T3_pred(x_teste_manual)[0])]
print("===  VOTAÇÃO EM x_teste = (2.5, 3.5) ===")
print(f"Voto T1: {votos_teste[0]} | Voto T2: {votos_teste[1]} | Voto T3:␣
↪{votos_teste[2]} => Decisão Majoritária Final: Classe {mode(votos_teste,␣
↪keepdims=False).mode}\n")
print("=== MATRIZ DE VALIDAÇÃO OOB ===")
print(f"{'Amostra':<10} | {'Alvo y':<6} | {'Árvores OOB':<12} | {'Voto OOB':<8}␣
↪| {'Acerto?'}")
print("-" * 55)
acertos_oob, total_avaliadas = 0, 0
for i in range(5):
    arvores_oob = oob_de_cada_amostra[i]
    if len(arvores_oob) == 0:
        print(f"{nomes_A[i]:<10} | {y_manual[i]:<6} | {'Nenhuma':<12} | {'-':
↪<8} | (Ignorada)")
    else:
        nomes_arv = ",".join([nome for nome, _ in arvores_oob])
        voto = int(mode([fn(X_manual[i:i+1])[0] for _, fn in arvores_oob],␣
↪keepdims=False).mode)
        acertou = (voto == y_manual[i])
        acertos_oob += int(acertou)
        total_avaliadas += 1
        print(f"{nomes_A[i]:<10} | {y_manual[i]:<6} | {'{' + nomes_arv + '}':
↪<12} | {voto:<8} | {'Sim' if acertou else 'Não'}")
print(f"--> Acurácia Global Out-of-Bag (Acc_OOB) = {acertos_oob}/
↪{total_avaliadas} = {acertos_oob/total_avaliadas:.2%}\n")
fig, axes = plt.subplots(1, 3, figsize=(14, 4))
gx, gy = np.meshgrid(np.linspace(0, 6, 150), np.linspace(0, 6, 150))
g_pts = np.c_[gx.ravel(), gy.ravel()]
dados_plot_arvores = [
    ("Árvore T1: $X_1 + X_2 \\leq 4.0$\n(OOB: A5)", T1_pred, [4]),
    ("Árvore T2: $X_1 \\leq 2.5$\n(OOB: A1)", T2_pred, [0]),
    ("Árvore T3: $X_2 \\leq 3.0$\n(OOB: A3, A4)", T3_pred, [2, 3])
```

```
]
for ax, (titulo, fn_pred, idx_oob) in zip(axes, dados_plot_arvores):
    ax.contourf(gx, gy, fn_pred(g_pts).reshape(gx.shape), alpha=0.25,␣
 ↪cmap='winter')
    ax.contour(gx, gy, fn_pred(g_pts).reshape(gx.shape), levels=[0.5],␣
 ↪colors='crimson', linewidths=2, linestyles='--')
    for i in range(5):
        cor_ponto = 'navy' if y_manual[i] == 0 else 'darkgreen'
        if i in idx_oob:
            ax.scatter(X_manual[i, 0], X_manual[i, 1], facecolors='none',␣
 ↪edgecolors='crimson', s=110, linewidths=2, linestyle='--')
            ax.text(X_manual[i, 0] + 0.15, X_manual[i, 1] - 0.25,␣
 ↪f"{nomes_A[i][:2]}(OOB)", fontsize=8.5, color='crimson', fontweight='bold')
        else:
            ax.scatter(X_manual[i, 0], X_manual[i, 1], color=cor_ponto,␣
 ↪edgecolors='k', s=80, zorder=4)
            ax.text(X_manual[i, 0] + 0.15, X_manual[i, 1] + 0.12, nomes_A[i][:
 ↪2], fontsize=8.5, fontweight='bold')
    ax.scatter([2.5], [3.5], color='gold', edgecolors='black', marker='*',␣
 ↪s=220, zorder=6, label='$x_{teste}(2.5, 3.5)$')
    ax.set_title(titulo, fontweight='bold', fontsize=10)
    ax.set_xlim(0, 6); ax.set_ylim(0, 6)
    ax.set_xlabel("$X_1$"); ax.set_ylabel("$X_2$")
    ax.legend(loc='upper left', fontsize=8)
    ax.grid(True, linestyle=':', alpha=0.6)
plt.tight_layout()
plt.show()
```

```
===  VOTAÇÃO EM x_teste = (2.5, 3.5) ===
Voto T1: 1 | Voto T2: 0 | Voto T3: 1 => Decisão Majoritária Final: Classe 1

=== MATRIZ DE VALIDAÇÃO OOB ===
Amostra    | Alvo y | Árvores OOB  | Voto OOB | Acerto?
-----------------------------------------------------
A1(1,2)    | 0      | {T2}         | 0        | Sim
A2(2,1)    | 0      | Nenhuma      | -        | (Ignorada)
A3(3,4)    | 1      | {T3}         | 1        | Sim
A4(4,3)    | 1      | {T3}         | 0        | Não
A5(5,5)    | 1      | {T1}         | 1        | Sim
--> Acurácia Global Out-of-Bag (Acc_OOB) = 3/4 = 75.00%
```

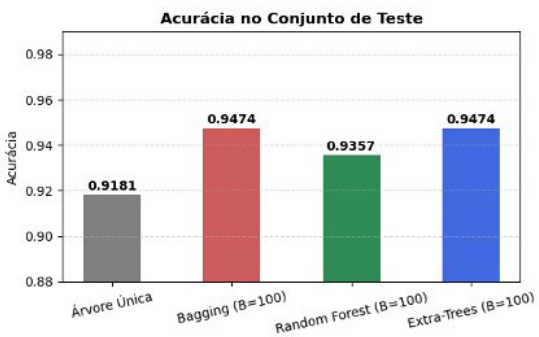


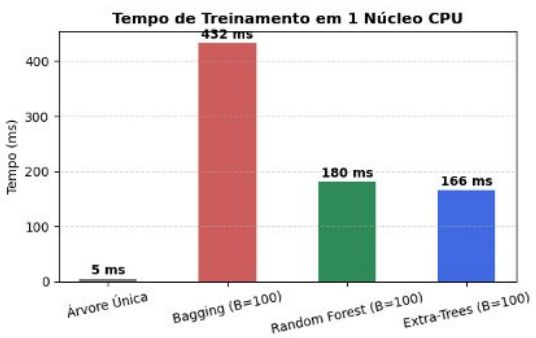
[28]:
```
modelos_benchmark = {
    "Árvore Única": DecisionTreeClassifier(random_state=42),
    "Bagging (B=100)": BaggingClassifier(n_estimators=100, oob_score=True,␣
 ↪random_state=42, n_jobs=1),
    "Random Forest (B=100)": RandomForestClassifier(n_estimators=100,␣
 ↪max_features='sqrt', oob_score=True, random_state=42, n_jobs=1),
    "Extra-Trees (B=100)": ExtraTreesClassifier(n_estimators=100,␣
 ↪max_features='sqrt', bootstrap=True, oob_score=True, random_state=42,␣
 ↪n_jobs=1)
}
nomes_b, accs_b, tempos_b = [], [], []
print(f"{'Modelo':<24} | {'Acc Treino':<10} | {'Acc Teste':<10} | {'OOB Score':
 ↪<10} | {'Tempo (ms)':<10}")
print("-" * 76)
for nome, mod in modelos_benchmark.items():
    t0 = time.perf_counter()
    mod.fit(X_tr, y_tr)
    dt_ms = (time.perf_counter() - t0) * 1000.0
    a_tr = accuracy_score(y_tr, mod.predict(X_tr))
    a_te = accuracy_score(y_te, mod.predict(X_te))
    oob_v = getattr(mod, 'oob_score_', None)
    oob_s = f"{oob_v:.4f}" if oob_v is not None else "N/A"
    nomes_b.append(nome); accs_b.append(a_te); tempos_b.append(dt_ms)
    print(f"{nome:<24} | {a_tr:<10.4f} | {a_te:<10.4f} | {oob_s:<10} | {dt_ms:
 ↪<10.1f}")

fig, axes = plt.subplots(1, 2, figsize=(12, 3.8))
b_acc = axes[0].bar(nomes_b, accs_b, color=['gray', 'indianred', 'seagreen',␣
 ↪'royalblue'], width=0.48)
axes[0].set_ylim(0.88, 0.99)
axes[0].set_title("Acurácia no Conjunto de Teste", fontweight='bold')
axes[0].set_ylabel("Acurácia")
axes[0].tick_params(axis='x', rotation=12)
for b in b_acc:
```

```
    h = b.get_height()
    axes[0].text(b.get_x() + b.get_width()/2., h + 0.002, f"{h:.4f}",␣
 ↪ha='center', fontweight='bold')
axes[0].grid(axis='y', linestyle='--', alpha=0.5)
b_tmp = axes[1].bar(nomes_b, tempos_b, color=['gray', 'indianred', 'seagreen',␣
 ↪'royalblue'], width=0.48)
axes[1].set_title("Tempo de Treinamento em 1 Núcleo CPU", fontweight='bold')
axes[1].set_ylabel("Tempo (ms)")
axes[1].tick_params(axis='x', rotation=12)
for b in b_tmp:
    h = b.get_height()
    axes[1].text(b.get_x() + b.get_width()/2., h + max(tempos_b)*0.02, f"{h:.
 ↪0f} ms", ha='center', fontweight='bold')
axes[1].grid(axis='y', linestyle='--', alpha=0.5)
plt.tight_layout()
plt.show()
```

Modelo	Acc Treino	Acc Teste	OOB Score	Tempo (ms)
Árvore Única	1.0000	0.9181	N/A	4.9
Bagging (B=100)	1.0000	0.9474	0.9548	432.4
Random Forest (B=100)	1.0000	0.9357	0.9573	180.1
Extra-Trees (B=100)	1.0000	0.9474	0.9724	165.9





[ ]: